



Tutorial S1C: Sorting Algorithms (Solutions)

Section A (Discussion Questions)

1. (a)

N	lst	array	passes	smallest	item	item < smallest
		[56,34,24,50,43]				
5	[]		0	56	56	False
					34	True
				34	24	True
				24	50	False
					43	False
	[24]	[56,34,50,43]	1	56	56	False
					34	True
				34	50	False
					43	False
	[24,34]	[56,50,43]	2	56	56	False
					50	True
				50	43	True
				43		
	[24,34,43]	[56,50]	3	56	56	False
					50	True
				50		
	[24,34,43,50]	[56]				
	[24,34,43,50,56]	[]				

(b) Insertion sort: (explain with characteristic of insertion sort) After each pass, the smallest number in the unsorted part is inserted into the correct position of the sorted part.

(c) Not in-place. Extra memory (lst) is used in the sorting process.

(d,e) In this algorithm, both best case and worst case time complexities are the same: $O(N^2)$. Outer for-loop runs $N-1$ times. For each iteration of outer for-loop, inner for-loop runs N times, then $N-1$ times, then $N-2$ times, etc, until 2 times. Therefore by summing up the number of iterations of inner for-loop: $N+(N-1)+(N-2)+\dots+2$ times, using summation of Arithmetic Progression formula, total inner loop runs $\frac{N-1}{2}(N+2)$, and highest power is 2, hence $O(N^2)$

2. (a) Algorithm 2 will more likely sort faster. The Noswap flag when true will exit the function. It will be true when the array is already sorted before the comparisons of all the iterations are completed.

(b) Algorithm 1: $O(N^2)$. Algorithm 2: $O(N)$ for best case, $O(N^2)$ for average to worst case.



```

3. def InsertionSortRec(Arr, N):
    if N > 0: #N not yet reach base case of 0
        InsertionSortRec(Arr, N-1) #parameter N-1 tends towards base case of 0
        unsortedValue = Arr[N]
        current = N-1
        while current >= 0 and unsortedValue < Arr[current]:
            Arr[current+1] = Arr[current]
            current = current - 1
        Arr[current+1] = unsortedValue #place the unsortedValue in its
                                     correct position in the sorted part
    return Arr
return Arr #base case: when N=0

```

4.

SplitPoint	First	Last	PivotValue	LeftMark	RightMark	Done	Arr
	0	6					[9,5,8,12,3,7,11]
			9	1	6	False	
				2			
				3			
					5		[9,5,8,7,3,12,11]
				4			
				5	4	True	[3,5,8,7,9,12,11]
4							

5. Insertion sort on 13, 3, 7, 11, 2.

Action

Assume 1st value from the left is sorted:

Compare 3 with those before (to the left) to find its position to insert:

Compare 7 with those before to find its position to insert:

Compare 11 with those before to find its position to insert:

Compare 2 with those before to find its position to insert:

Result

13

3, 13

3, 7, 13

3, 7, 11, 13

2, 3, 7, 11, 13

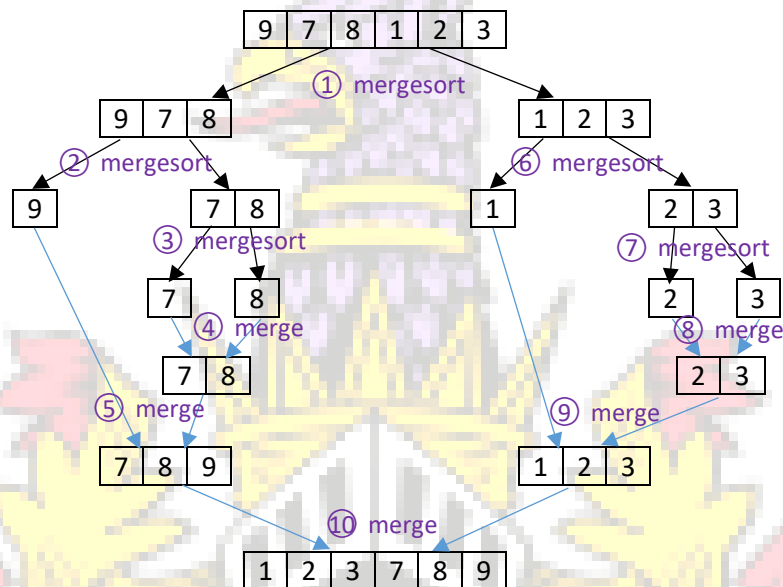
6.

List A	List B
3, 7, 11, 13	1, 4, 8, 10, 16, 18, 19
Compare	New list
List A: 3, List B: 1	1
List A: 3, List B: 4	1, 3
List A: 7, List B: 4	1, 3, 4
List A: 7, List B: 8	1, 3, 4, 7
List A: 11, List B: 8	1, 3, 4, 7, 8
List A: 11, List B: 10	1, 3, 4, 7, 8, 10
List A: 11, List B: 16	1, 3, 4, 7, 8, 10, 11
List A: 13, List B: 16	1, 3, 4, 7, 8, 10, 11, 13
List A is empty, copy remaining from list B into new list:	1, 3, 4, 7, 8, 10, 11, 13, 16, 18, 19

- Compare first number from each list
- Smallest is put into new list and the number replaced by next one from its original list
- Repeat until one list becomes empty
- When one list becomes empty, copy remaining numbers from the other list into new list.



7.



Number in circle shows the order of calling functions mergesort and merge.

8. Function calls made in chronological order:

No.	Function call		
1	mergeSort(ar,0,7)		
2		mergeSort(ar,0,3)	
3			mergeSort(ar,0,1)
4			mergeSort(ar,0,0)
5			mergeSort(ar,1,1)
6			merge(ar,0,0,1)
7			mergeSort(ar,2,3)
8			mergeSort(ar,2,2)
9			mergeSort(ar,3,3)
10			merge(ar,2,2,3)
11		merge(ar,0,1,3)	
12		mergeSort(ar,4,7)	
13			mergeSort(ar,4,5)
14			mergeSort(ar,4,4)
15			mergeSort(ar,5,5)
16			merge(ar,4,4,5)
17			mergeSort(ar,6,7)
18			mergeSort(ar,6,6)
19			mergeSort(ar,7,7)
20			merge(ar,6,6,7)
21		merge(ar,4,5,7)	
22	merge(ar,0,3,7)		

Total number of times mergeSort is called (including the first call) = 15



9. **Comparison of Methods**

Consider these criteria when choosing a sorting method, assume random dataset.

- ☞ Use of space
- ☞ Use of computer time
- ☞ Programming effort

a. Use of space

This concerns the **size of the list**, the **memory space** and even the **number of disk drives** available (for external sorting).

b. Use of computer Time

This concerns the **number of comparisons** and the **number of swaps** for each sorting method.

c. Programming Effort

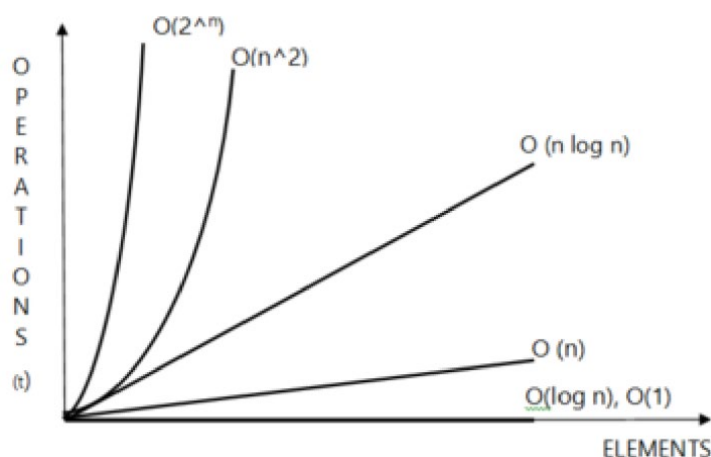
This concerns the effort to write the program. Consider the big O of each algorithm, if the running time of an algorithm is only marginally better, or if a program is to be run only once or twice, then it may not be wise for a programmer to spend a long time investigating many sophisticated algorithms that might, in the end, only save a few seconds of computer time.

10. $O(n \log n)$. $1024 \log 1024 = 3082.5$

$O(n^2)$. $1024^2 = 1048576$

Difference factor = 340.

Therefore $O(n^2)$ takes a longer time, a factor of 340 times longer.



From the graph, it can be seen that rate of increase of $O(n^2)$ is faster than $O(n \log n)$.



Section B (Assignment Questions)

1. Both sort algorithms have time complexity of $O(n^2)$.
However, bubbleSort1 will require less iterations of the array as compared to bubbleSort2. In general, after each pass in bubble sort, the right-most element of the unsorted part of the array will be sorted. With the *passes* variable in bubbleSort1, the algorithm **does not iterate** over the sorted part of the array, whereas in bubbleSort2 it **iterates over** the sorted part of the array in each pass. Thus, bubbleSort1 sorts faster.
2. Even though the compare operations are fundamentally similar, swap operations in 1 & 2 **are very different**. Swap operation in 1 essentially involves "**shifting**" elements in the sorted side of the array to the right, then inserting the unsorted value into the correct position in the sorted side of the array. Swap operations in 2 involve **continuously swapping** unsorted value with values in sorted side of array until it arrives in its correct position.
Swap operation in 1 is faster than swap operation in 2, as **continuously swapping** elements in 2 involves extra steps of swapping. A shift of an element in the swap operation of insertionSort1 can be thought of as moving one element, whereas a swap of two elements in the swap operation of insertionSort2 can be thought of moving two elements.

3. A single function mergesort.

```
FUNCTION mergesort(arr: ARRAY[1:N] of INTEGERS) RETURNS ARRAY
    IF LENGTH(ARR) = 1 THEN
        RETURN ARR
    mid = INT((1+N)/2)
    leftList = mergesort(arr[1:mid])
    rightList = mergesort(arr[mid:N])
    i = 0
    j = 0
    k = 0
    WHILE i < LENGTH(leftList) AND j < LENGTH(rightList) DO
        IF leftList[i] < rightList[j] THEN
            arr[k] = leftList[i]
            i = i+1
        ELSE
            arr[k] = rightList[j]
            j = j+1
        ENDIF
        k = k+1
    ENDWHILE
    WHILE i < LENGTH(leftList) DO
        arr[k] = leftList[i]
        i = i+1
        k = k+1
    ENDWHILE
    WHILE j < LENGTH(rightList) DO
        arr[k] = rightList[j]
        j = j+1
        k = k+1
    ENDWHILE
    RETURN arr
ENDFUNCTION
```